

Laboratorio 3: Algoritmos de Enrutamiento (Parte 2)

Informe de pruebas con Flooding, DVR y LSR sobre Redis

Sección 10 – Grupo 7

2025-08-27

Resumen

Se documenta la implementación y prueba de **Flooding**, **Distance Vector (DVR)** y **Link State Routing (LSR)** empleando **Redis** (pub/sub) como medio de transporte. Se siguen las directrices del laboratorio respecto al formato de mensajes, comandos de operación y evidencia experimental de entrega y reconvergencia. Se adjuntan capturas de pantalla y comandos para reproducibilidad.

1. Antecedentes y objetivos

El objetivo es analizar, instrumentar y validar tres enfoques de enrutamiento: difusión (Flooding), vector distancia (DVR) y estado de enlace (LSR), demostrando *respeto de saltos, conectividad end-to-end y adaptación a fallos*.

2. Arquitectura y protocolo

El proyecto expone un nodo (Node) con transporte Redis, bucles de recepción, reenvío y cómputo de rutas, más un *shell* con comandos: **send**, **broadcast**, **ping**, **show routes**, **show neighbors**, **show lsdb**, **show dv**. El formato de mensaje usado para interoperar es:

```
{
  "type": "hello|message|info|echo",
  "from": "sec10.grupoX.alumno",
  "to": "sec10.grupoY.alumno",
  "hops": 0,
  "headers": [{"alg":"lsr|dvr|flooding"}, {"ttl":8}, {"id":"..."}],
  "seq_num": 0,                // solo en INFO (LSR)
  "neighbors": {"B":1.0},      // solo en INFO (LSR)
  "payload": "texto"           // solo en MESSAGE
}
```

3. Metodología de pruebas

Una terminal por nodo, con `run_demo.py` y banderas: `-names`, `-topo`, `-transport redis`, `-alg {flooding,lsr,dvr}`, `-src`, `-shell`. Tras 3–5 s de *warmup* se ejecutan `send`, `show routes`, `show neighbors`, `show lsdb/show dv`.

4. Resultados

Prueba 1: Topología en línea (Flooding)

A-B-C-D-E. Con todos los nodos activos, A→E se entrega. Al detener C, no hay ruta alternativa y el mensaje no se entrega (confirmación de que no se “saltan” vecinos).



Figura 1: Topología en línea con Flooding.

Prueba 2: Topología en anillo (LSR)

A-B-C-D-E-F-G-A. A→F usa el camino mínimo por G. Al apagar G, LSR reconverge y la ruta pasa por B-C-D-E manteniendo la entrega.



Figura 2: Anillo con LSR: adaptación ante la caída de G.

Prueba 3: Distance Vector (DVR) en línea (A-B-C)

Objetivo. Mostrar convergencia de DVR y su reacción a fallos en una topología mínima.

Procedimiento.

1. Topología: { A: [B], B: [A, C], C: [B] } con costos unitarios.
2. Arranque de tres nodos con `-alg dvr` (una terminal por nodo).
3. Esperar a la *convergencia*: en cada nodo, `show routes` y `show dv` deben reflejar el costo mínimo y el *next hop* correcto hacia los otros dos.
4. Enviar `send C "hola DVR"` desde A para validar entrega A→C.
5. *Fallo*: cerrar la terminal de B. Tras el *timeout* de vecindad, `show routes` debe indicar que C es inalcanzable desde A; reintentar `send C "..."` y constatar que no se entrega.

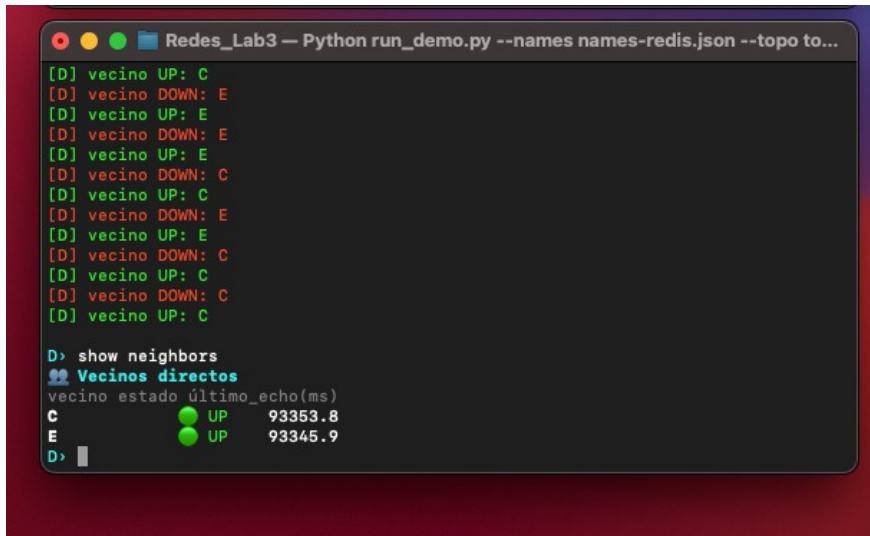
Comandos (ejemplo).

```
python3 run_demo.py --names names-redis.json --topo topo-linea-3.json \
  --transport redis --alg dvr --src A --shell \
  --redis-host lab3.redesuvlg.cloud --redis-port 6379 \
  --redis-user default --redis-pass UVGRedis2025
# ... repetir para B y C
# En A: show dv; show routes; send C "hola DVR"
# Apagar B; esperar ~5-10 s; show dv; show routes; reintentar envío
```

Observaciones. En nuestra implementación, DVR intercambia periódicamente vectores de distancia con los vecinos y actualiza la tabla local por *Bellman-Ford hop-by-hop*. El experimento evidencia (i) convergencia a costos mínimos y (ii) invalidación de rutas cuando cae un enlace crítico en una topología en línea.

Diagnóstico (neighbors y lsdv/dv)

`show neighbors` reporta estado UP/DOWN y latencias de ECHO; `show lsdv` (LSR) muestra la base de enlaces; `show dv` (DVR) lista el vector distancia. A modo ilustrativo se incluyen capturas reales:



```
[D] vecino UP: C
[D] vecino DOWN: E
[D] vecino UP: E
[D] vecino DOWN: E
[D] vecino UP: E
[D] vecino DOWN: C
[D] vecino UP: C
[D] vecino DOWN: E
[D] vecino UP: E
[D] vecino DOWN: C
[D] vecino UP: C
[D] vecino DOWN: C
[D] vecino UP: C
```

```
D> show neighbors
Vecinos directos
vecino estado último_echo(ms)
C          ● UP      93353.8
E          ● UP      93345.9
D>
```

Figura 3: `show neighbors`: vecindad y último ECHO (ms).

```
Redes_Lab3 - Python run_demo.py --names names-redis.json --topo to...
[D] vecino UP: E
D> show lsdb
Link-State DB (LSR)
u      v      costo
A      B      1.0
A      G      1.0
B      A      1.0
B      C      1.0
C      B      1.0
C      D      1.0
D      C      1.0
D      E      1.0
E      D      1.0
E      F      1.0
F      E      1.0
F      G      1.0
G      A      1.0
G      F      1.0
D>
```

Figura 4: show lsdb: enlaces y costos unitarios (LSR).

5. Conclusiones

- **Flooding** entrega mientras haya conectividad; no “salta” nodos.
- **LSR** calcula y actualiza caminos mínimos globalmente; se observó reconvergencia ante la caída de G en anillo.
- **DVR** converge por intercambio de vectores con vecinos y marca inalcanzables tras fallos; la prueba A–B–C demostró convergencia y pérdida de alcance al caer B.

Anexo: Comandos rápidos

Flooding (línea A–B–C–D–E)

```
python3 run_demo.py --names names-redis.json --topo topo-linea-5.json \
  --transport redis --alg flooding --src A --shell \
  --redis-host lab3.redesuvlg.cloud --redis-port 6379 \
  --redis-user default --redis-pass UVGRedis2025
# ... repetir para B,C,D,E
```

LSR (anillo A–...–G)

```
python3 run_demo.py --names names-redis.json --topo topo-anillo-7.json \
  --transport redis --alg lsr --src A --shell \
  --redis-host lab3.redesuvlg.cloud --redis-port 6379 \
  --redis-user default --redis-pass UVGRedis2025
# ... repetir para B..G
```

DVR (línea A–B–C)

```
python3 run_demo.py --names names-redis.json --topo topo-linea-3.json \
  --transport redis --alg dvr --src A --shell \
  --redis-host lab3.redesuvlg.cloud --redis-port 6379 \
  --redis-user default --redis-pass UVGRedis2025
# ... B y C (cambiando --src)
```